Objectif et modèle conceptuel

1. Objectif et périmètre

Le moteur transforme un document source de Modules et de Nodes en un **AST logique indépendant des langages cibles**, puis en informations sémantiques et diagnostics. Il accepte les étapes intermédiaires de construction : une erreur ne détruit ni la source ni la possibilité d'inspecter l'arbre.

Le périmètre livré couvre la construction de l'AST, les portées lexicales, les références locales et intermodules, un typage statique strict et la sérialisation JSON versionnée. S'y ajoutent, sur action explicite seulement, une **simulation locale** déterministe, un **débogage pas à pas**, des **contrats de nœud** vérifiés pendant l'exécution, une **analyse de graphes** (dépendances, flot de contrôle, accessibilité) et une **transpilation** vers JavaScript et Python. Le moteur est du JavaScript pur, sans React, DOM, Neutralinojs ou dépendance npm ; l'interface graphique est désormais branchée à ce moteur.

Exclus : suggestions contextuelles, exécution automatique, génération de code implicite, analyse de flot avancée, modèles objets et persistance propre du moteur. Le document décrit des fondations, pas un compilateur complet.

2. Concepts et relations distinctes

Project	Document source portant un identifiant, une version de format, une liste de modules et une liste globale de nœuds.
Module	Unité logique avec identifiant stable, racines syntaxiques, imports et exports explicites. Aucun lien obligatoire avec un fichier physique.
Node	Enregistrement discriminé par `kind`, propriétaire `moduleId`, propriétés, enfants syntaxiques et références symboliques.
AST	Arborescence dérivée : les identifiants d'enfants sont remplacés par des constructions hiérarchiques. Les `sourceId` restent stables.
Scope	Portée lexicale avec parent ; elle ne correspond pas nécessairement au module propriétaire.
Symbol	Déclaration accessible depuis une portée, ou alias d'import, associé à son nœud et à un type statique résolu ; un échec reste diagnostiqué sans type de secours.
Contract	Contrainte déclarative optionnelle (`positive`, `nonNegative`, `nonEmpty`) attachée à une variable, une constante ou un paramètre et évaluée pendant l'exécution.

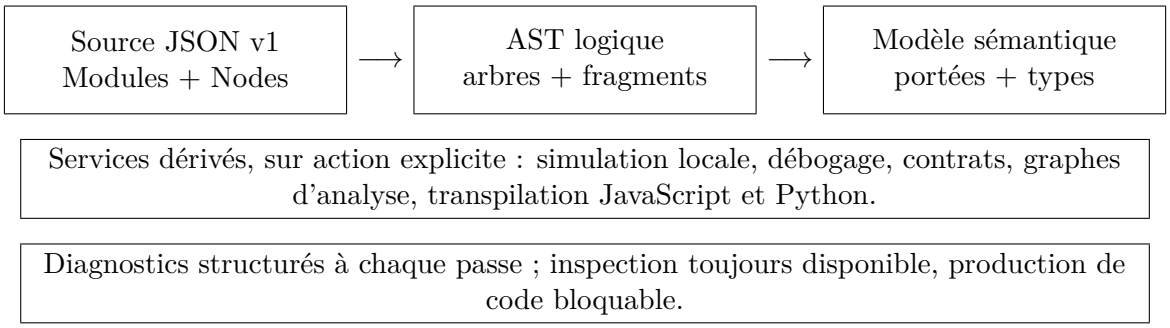
Trois relations indépendantes : appartenance par `moduleId`, imbrication par `children`, référence par `references.target`. Une référence vers une déclaration ne la transforme pas en enfant de l'appelant. Une connexion graphique n'a donc aucun sens logique implicite.

L'ancienne catégorie source `main` est retirée. Un projet valide contient zéro ou une seule fonction portant `properties.entryPoint: true`, indépendamment de son nom ; toute désignation supplémentaire produit `ENTRY_POINT_DUPLICATE` et un marqueur mal placé `ENTRY_POINT_INVALID`, sans suppression des données invalides. Aucun nom, signature cible ni appel implicite n'est imposé : la simulation, le débogage et la transpilation n'agissent que sur un lancement explicite. Les sous-modules

ne sont pas introduits à ce stade.

Architecture et invariants

3. Flux et responsabilités



`model.js`, `nodeSchema.js`	Contrats JSDoc, fabriques sans génération d'identifiant, catégories, slots et opérateurs centralisés.
`ast.js`	Indexation, reconstruction des enfants, contrôle de structure, cycles, parents multiples et propriétaires ; parcours et fragments détachés.
`semantics.js`	Collecte des portées et déclarations, contrôle des imports/exports et résolution des références.
`typeAnalysis.js`, `primitives.js`	Annotations obligatoires, signatures, tableaux et expressions statiquement déterminées ; catalogue des dix primitifs, bornes et arrondis.
`numericConstants.js`	Contrôles des expressions constantes : conversions, dépassements, sous-flux et division par zéro.
`diagnostics.js`, `serialization.js`	Codes stables, gravité, localisation ; lecture et écriture JSON sans migration implicite.
`sourceMutation.js`, `nodeFactory.js`, `actionResolver.js`, `contextResolver.js`	Édition contextuelle atomique : actions disponibles, cache d'analyse, insertion, mise à jour, suppression et réordonnancement.
`simulation.js`, `contracts.js`	Interpréteur local déterministe, trace, valeurs observées et contrats vérifiés après chaque instruction.
`debugging.js`, `reachability.js`	Session de débogage déterministe (états préparés puis parcourus) et analyse conservatrice d'accessibilité.
`nodeGraphs.js`, `moduleLinks.js`	Graphes de dépendances et de flot de contrôle d'un sous-arbre ; liens intermodules réellement utilisés.
`nodeTransfer.js`, `moduleInterface.js`, `deadCode.js`, `deletionImpact.js`	Transferts de sous-arbres, exports/imports de module, déclarations inutilisées et impact d'une suppression.

Ces fichiers sont dans `src/utils/logic/` ; l'API publique est `index.js`. La fonction `analyzeProgram(source)` renvoie `source`, `ast`, `semantic`, `diagnostics` et `canGenerate`, sans jamais modifier la source. Les backends de transpilation vivent séparément dans `src/utils/transpilation/` ; JavaScript et Python sont actifs, C, C++ et Java restent indisponibles.

4. Structures et invariants

- Identifiants de nœuds uniques dans le projet ; identifiants d'import uniques dans leur module. Aucun identifiant n'est recréé pendant les passes.
- Un enfant syntaxique a au plus un parent et appartient au même module. L'ordre des listes de slots est significatif, pas celui du tableau `nodes`.
- Déclarations : variables, constantes, fonctions, paramètres et alias de types. Instructions : blocs, retours, affectations, conditions, boucles (`break`/`continue` compris), instructions d'expression et affichage. Expressions : littéraux, références, appels, opérations unaires/binaires, littéraux et créations de tableaux, accès à un élément et conversions.
- Un slot requis absent produit `Hole`. Un lien impossible produit `Invalid`. Les fragments détachés restent visibles ; la source conserve également les enregistrements malformés ou dupliqués.
- Annotations obligatoires des déclarations et littéraux : `primitive` ou alias transparent `named`. Signature `function` dérivée pour les fonctions. Dix primitifs : `short int`, `short uint`, `int`, `uint`, `float`, `double`, `string`, `char`, `boolean`, `void` (retour uniquement). Le type tableau `{kind:'array', elementType}` s'annote uniquement sur une variable, une constante ou un paramètre, avec un élément primitif non `void`. Aucune promotion ni conversion implicite.

Exemple et résolution des références

5. Deux modules, une référence intermodule

Le scénario exécutable `twoModules()` dans `tests/logic/fixtures.js` crée la constante publique `answer = 42` dans `math`, puis la variable `result` dans `app`.

```
Project { formatVersion: 1, id: "project" }
  Module "math"
    rootNodeIds: ["answer"]
    exports: [{ nodeId: "answer", name: "answer" }]
  Module "app"
    rootNodeIds: ["result"]
    imports: [{ id: "answerImport", moduleId: "math",
                nodeId: "answer", localName: "answer" }]

Node "answer" : ConstantDeclaration, moduleId = "math"
  properties: { name: "answer", type: primitive("int") }
  children.initializer: "fortyTwo"
Node "fortyTwo" : Literal, value = 42, type = primitive("int")

Node "result" : VariableDeclaration, moduleId = "app"
  properties: { name: "result", type: primitive("int") }
  children.initializer: "answerReference"
Node "answerReference" : Reference, moduleId = "app"
  references.target: { kind: "import", importId: "answerImport" }
```

`primitive("int")` est ici une notation abrégée pour `{kind:"primitive", name:"int"}`.

L'AST simplifié est le suivant ; chaque construction conserve son identifiant source :

```
Project
  Module math
    ConstantDeclaration [answer]
      initializer -> Literal [fortyTwo] : int
  Module app
    VariableDeclaration [result]
      initializer -> Reference [answerReference] : int
```

La déclaration `answer` n'est pas recopiée sous `result`. Le modèle sémantique associe la référence au symbole importé et au nœud cible `answer`. Sans export ou import explicite, la référence est rejetée.

Règles de portée : une portée par module, fonction, bloc imbriqué et `for`. Paramètres et bloc principal d'une fonction partagent une portée. Les noms sont collectés avant résolution ; un doublon local est une erreur, un masquage dans une portée interne est autorisé. Une référence directe par identifiant ne contourne pas les frontières lexicales ou les imports.

Vérification et évolutivité

6. Validation et résultats constatés

Un diagnostic contient `code`, `severity`, `message` et `location = {projectId, moduleId, nodeId, field}`. La localisation désigne le nœud fautif ou le propriétaire d'un slot manquant. Les éléments inapplicables sont nuls. Les codes ne dépendent pas de la langue du message.

Contrôles structurels : identifiants invalides/dupliqués, enfant absent, mauvais propriétaire, catégories illégales, cycles, parents multiples, fragments détachés et slots incomplets. Contrôles sémantiques : référence inaccessible, import/export invalide, conflit de déclaration, point d'entrée dupliqué ou invalide, retour hors fonction, contrôle de boucle hors boucle, affectation d'une constante.

Typage strict : annotations, initialisation, opérateurs, conditions booléennes, signatures/appels, affectations et retours. Les tableaux (littéral, création, accès) doivent respecter le type d'élément annoncé ; les tableaux imbriqués et un alias nommé comme élément sont refusés. Les expressions non typables restent inspectables sans type publié. Le contrôle de retour accepte les blocs et les deux branches d'un `if`, mais ne prouve pas les retours via les boucles. Entiers signés/non signés de 16/32 bits, flottants IEEE 754 de 32/64 bits ; `char` est un scalaire Unicode. Les conversions `CastExpression` sont numériques explicites, avec troncature ou arrondi et contrôle des bornes.

Vérifications exécutées : `pnpm test`, runner natif Node : **285 tests réussis, 0 échec**. Couverture de la construction de l'AST, des portées et références, du typage et des tableaux, des bornes et arrondis numériques, de la sérialisation JSON, de la simulation (tableaux, contrats et fonction choisie compris), du débogage pas à pas et de l'accessibilité, des graphes de dépendances et de flot, de la transpilation JavaScript et Python, des transferts de sous-arbres, des exports/imports et de la détection des déclarations inutilisées.

`pnpm build` : frontend compilé avec succès. Ce contrôle protège le frontend existant ; le moteur séparé est vérifié par les tests Node, pas par la compilation du bundle. Le moteur reste sans React, DOM ni API native ; l'interface ne partage avec lui que les identifiants de module et les libellés. Aucune fonction native nouvelle n'a été ajoutée pour ce document.

7. Simulation, contrats et transpilation

Sur action explicite, `simulateProgram` exécute le point d'entrée — ou une fonction choisie avec des entrées typées — sans modifier la source. Le résultat reste en mémoire : sorties, trace ordonnée d'identifiants de nœuds, valeur de retour et variables observées. Un budget de pas (1 à 100 000) et une profondeur d'appel bornée (200) limitent le travail ; boucles non terminantes et appels trop profonds sont interrompus proprement. Les tableaux s'exécutent localement, avec contrôle des bornes et des cases jamais remplies.

Les contrats `positive`, `nonNegative` et `nonEmpty` attachés à une déclaration sont évalués après chaque instruction ; leurs violations sont signalées sans interrompre le programme. Le débogage réutilise le même interpréteur : les états sont préparés puis parcourus (suivant entrant dans les appels, sortie de scope, retour à l'arrêt précédent, redémarrage), sans exécution native suspendue.

La transpilation délègue à des backends enregistrés : JavaScript et Python sont actifs, C, C++ et Java restent indisponibles. Les diagnostics bloquants empêchent toute génération ; un point

d'entrée paramétré est refusé plutôt qu'appelé sans arguments. Le code produit n'est jamais exécuté automatiquement par l'application.

8. Évolution sans couplage prématuré

La simulation, le débogage et la transpilation consomment déjà l'AST et les symboles résolus. Les **suggestions contextuelles** au point d'édition et de **nouveaux générateurs** de cibles restent à venir ; chaque générateur devra refuser les diagnostics bloquants. `canGenerate` signifie seulement « aucune erreur détectée par les passes actuelles », et non une garantie d'exécutabilité. Aucun de ces moteurs n'est implémenté et aucun système de plugins n'a été inventé.

Le format v1 préserve les champs supplémentaires. Une version inconnue est conservée et signalée, sans migration. La sérialisation refuse les données non JSON au lieu de les supprimer silencieusement. Les profondeurs d'analyse et de sérialisation sont bornées pour éviter les dépassements de pile.

Limites assumées : pas de sous-modules, objets, génériques, réexports ni résolution de fichiers ; un tableau porte un élément primitif unique, sans tableau imbriqué. Les dépassements constants sont diagnostiqués ; les valeurs reçues à l'exécution ne sont pas bornées globalement par l'analyse. Les cibles C, C++ et Java sont annoncées mais indisponibles. Les futurs backends devront appliquer les mêmes opérations contrôlées, sans rebouclage. L'API complète et les conventions de slots sont décrites dans `docs/LOGIC_API.md`.